

PSU/IPMI tool documentation

By Hayden John Fortunata



1.0 Thesis

- 1.1 The first section of this document will be how to install the necessary drivers and download the executable.
- 1.2 The remainder sections of this documentation is to provide the customer with an understanding of the mathematical conversions of the raw IPMI data and why we have passed certain parameters into the Python script.
- 1.3 To gather the raw data for this document, I used a third-party tool called IPMIUTIL.exe, which brings over raw data by using the command “ipmiutil.exe sensor -r”

Note: The third-party program will not be used in the program; we will directly access the driver using Windows' built-in functions.

- 1.4 The referenced document throughout this process is the Intelligent Platform Management Specification Second Generation v2.0.
It can be found at this hyperlink: [IPMI-Spec](#)

2.0 Installing Necessary Drivers

- 2.1 First, we need to install the basic Intel chipset driver. This can found on the website for IMB530, [click here](#)

2021/12/28	Chipset driver	VA1.1	3.9 MB
------------	----------------	-------	--------

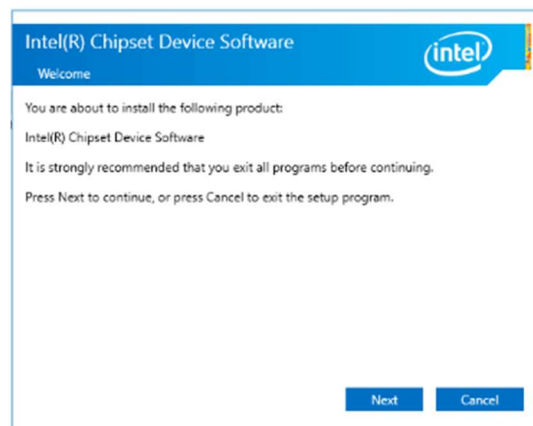
- 2.2 Once downloaded extract the folder

- 2.3 Under “\1. Intel Chipset Driver\1. Intel Chipset Driver\637610\Chipset-10.1.18698.8258-Public-MUP\” Click SetupChipset.exe

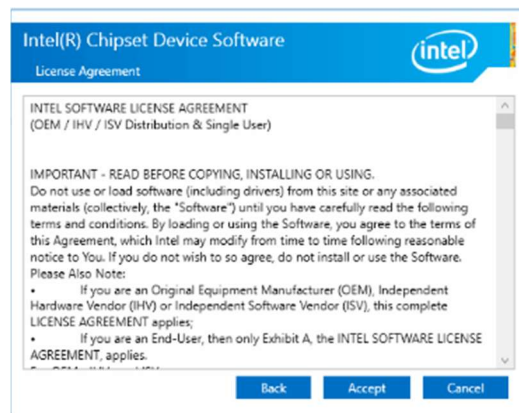
Name	Date modified	Type	Size
DriverFiles	4/28/2026 9:20 AM	File folder	
mup	4/28/2026 9:20 AM	Microsoft Edge H...	1,200 KB
SetupChipset	4/28/2026 9:20 AM	Application	2,954 KB
WixLicenseNote	4/28/2026 9:20 AM	Text Document	4 KB

- 2.4 Allow the app to make changes to your device

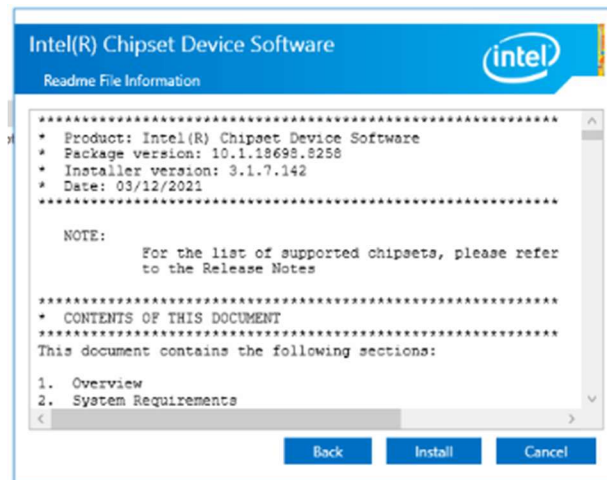
- 2.5 Click next



2.6 Click accept



2.7 Click Install

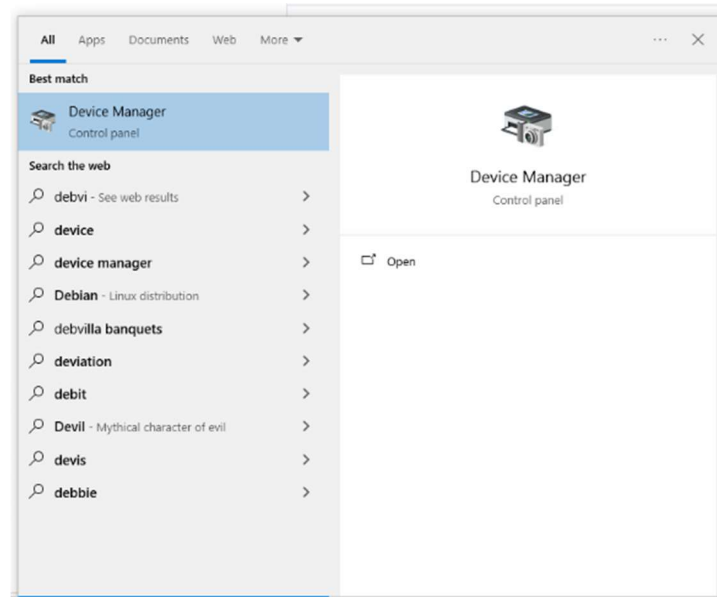


2.8 Allow installation to take place and click Finish.

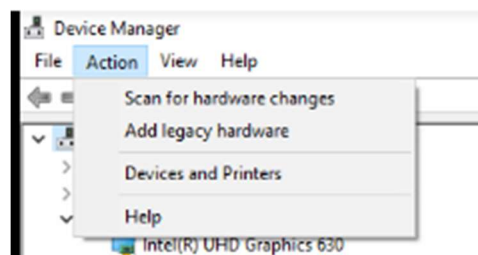
2.9 Reboot system after installation is done.

2.10 To communicate with the board, we need to install the basic IPMI driver used in Windows.

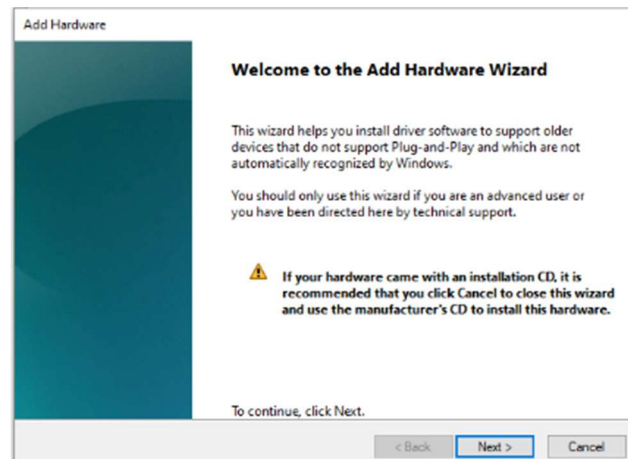
2.11 Go to Device Manager



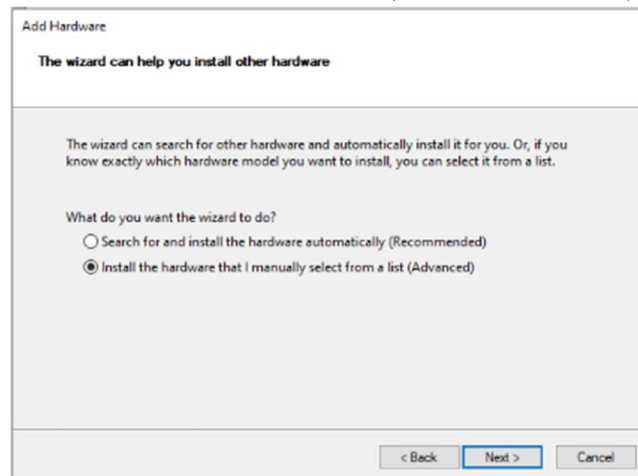
2.12 Under Action Select, Add legacy hardware.



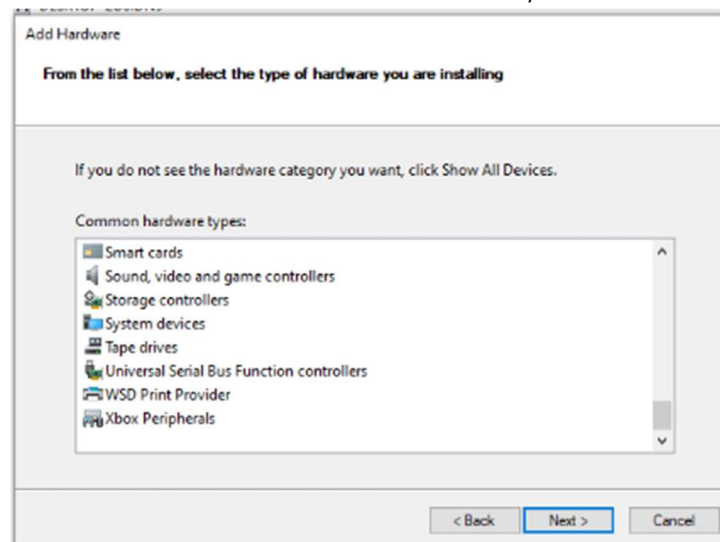
2.13 Click Next.



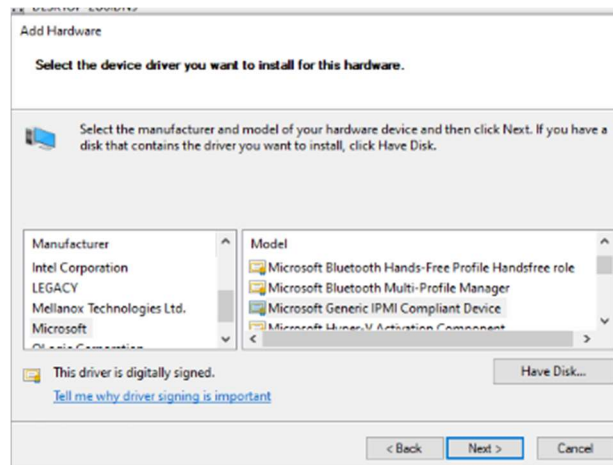
2.14 Click “Install the hardware that I manually select from a list(Advanced)”



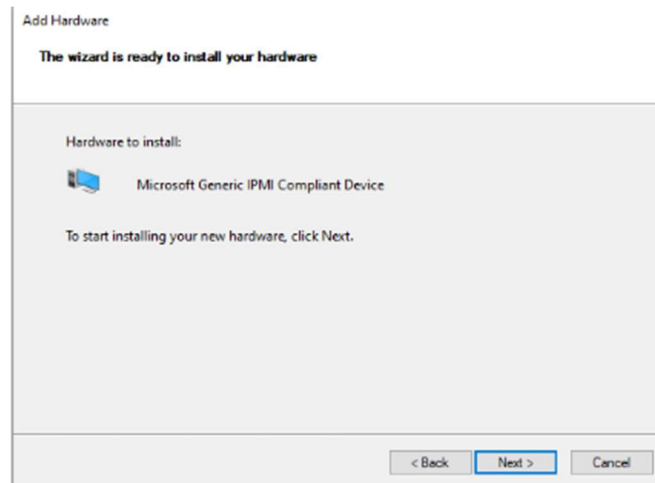
2.15 Scroll down to Add hardware and double click System Devices



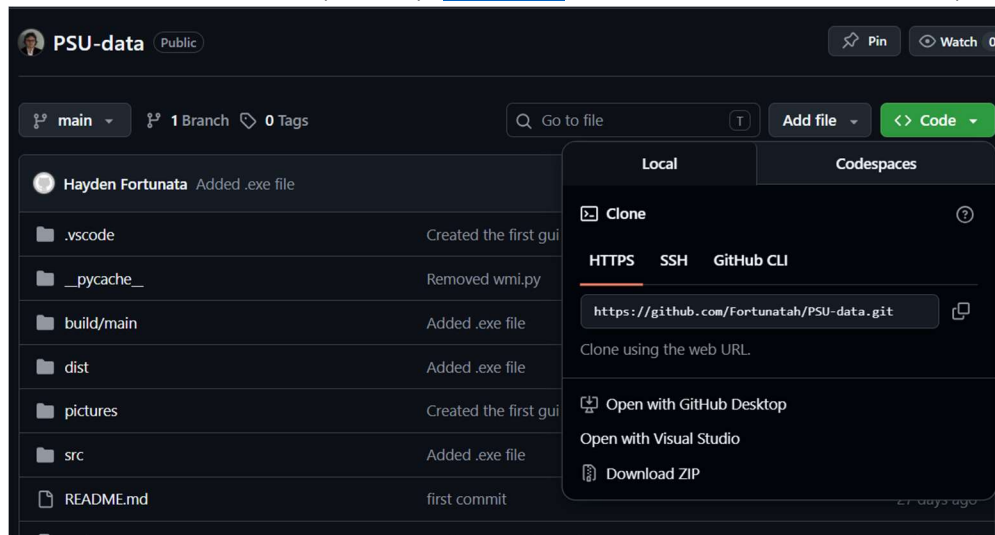
2.16 In the left pane under manufacturer select Microsoft, scroll down in the right pane under model and select “Microsoft Generic IPMI compliant device”.



2.17 Click Next.



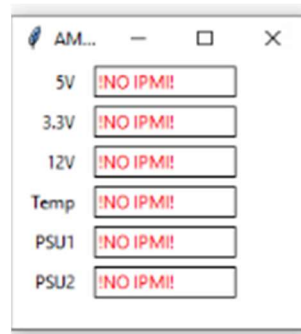
2.18 To download the repository, [click here](#), and click code-> Download Zip.



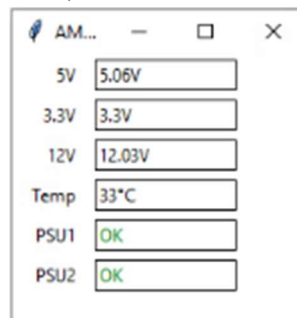
- 2.19 Extract the folder and navigate to “\PSU-data\PSU-IPMI.exe”, right click and say Run as Administrator.
- 2.20 If the “Windows protected your PC” prompt pops, click more info, and then Run.



- 2.21 If the program says “NO IPMI” in red, it means either the IPMI device is nonexistent, or the drivers were not installed properly. Additionally it could mean that program was not ran as Administrator.



- 2.22 The program will appear as below. The PSU-IPMI.exe can be moved out of the PSU-data directory and be put wherever the user desires.



3.0 IPMI response command

- 3.1 For us to receive the data, we need to be able to communicate with the BMC. In Python, there is a library called WMI. WMI allows us to communicate directly with the board.
- 3.2 In the block of code below is how the program grabs the raw data for each sensor.

```
def run_IPMI( address ):  
    result = ipmi.RequestResponse(  
        NetworkFunction=0x04,  
        Lun=0x00,  
        ResponderAddress=0x20,  
        Command=0x2D,  
        RequestDataSize=1,  
        RequestData=[address]  
    )  
  
    comp_code = result[0]  
    resp_data = result[1]  
    if comp_code == 0: return resp_data  
    else: return False
```

The request response will return three objects; we only need the first two

- 3.3 The first object *result[0]* is noted at *comp_code*, which is the completion code to see if the command successfully found the sensor and read it. If the completion code is 0, that means the response was valid.
- 3.4 If *comp_code* is 0, then we return the response data; if it is not 0, we return Boolean false.
- 3.5 Response data is captured from results as *result[1]*, in this object lies a tuple of integers; we need the second piece of data, as that is the sensor's raw data, so we return that data only.

- 3.6 We call this in the class *IPMI_sensors.refresh()*, we pass it the address of the sensor data we are trying to collect. If the value is false (the completion code is not 0), we send “!ERROR!” to the GUI; if it is valid, we process the data and perform the necessary operation on it.

```
IPMI_result = run_IPMI(0x01)
if not IPMI_result: self.vol3v3 = "!ERROR!"
else:
    self.vol3v3 = convert_to_volts(IPMI_result[1] , 172 , -4 )
```

4.0 Convert to Volts

- 4.1 For 3v3 volts, 5 volts, 12 volts, PSU 1 volts and PSU 2 volts, the raw data sent over needs to be converted using a formula found in the IPMI specification document. I will use 5 volts as my example.

36.3 Sensor Reading Conversion Formula

The following presents the formula used for converting ‘raw’ sensor readings for linear and linearized sensors to real values in the desired ‘units’ for the sensor (e.g. Volts, Amps, etc.).

$$y = L[(Mx + (B * 10^{K1})) * 10^{K2}] \text{ units}$$

Variable	Meaning
Y	The final answer
X	The raw reading
M	Multiplier(Bytes 25 and 26)
B	The Additive Offset(Bytes 27 and 28)
K1	The B exponent(Byte 30)
K2	The decimal shifter(Byte 30)
L	Linearization function(always 1 for volts)

- 4.2 Using the command “*ipmituile.exe sensor -r*”, we can get the necessary raw hex bytes for the equation above. Below is an example of the output of this command for 12 volts.

raw SDR: 02 00 51 01 3b 20 00 02 07 00 00 68 02 01 85 32 85 32 1b 1b 00 04 00 00 05 40 00 00 00 c0 00
00 00 00 00 00 00 cb c9 00 b4 b6 00 00 00 00 00 04 50 5f 35 56 00 00 00 00 00 00 00 00 00 00

- 4.3 The raw reading **X** will be collected from *result[1]*, as shown earlier.
- 4.4 The multiplier **M** is bytes 25 and 26, which in this case is 05 40. According to the IPMI specification, in 10-bit IPMI math, byte 25 is the low part, which is 5. For byte 26, we take the top 2 bits for 10-bit IPMI math.
The binary of 40(hex) is *01000000*. The top 2 bits are *01*. We then add this to the front of our low part, converted to binary math. 05 in binary 8-bit form is *00000101*.

$$01 + 00000101 = 010000101 = 261.$$

Leading us to the conclusion of **M = 261**.

- 3.3 For the additive offset **B**, we collect bytes 26 and 27, which in this case are 00 and 00. This will be the same for voltage readings in this program.

From this logic, in our equation for voltage conversion, we can get rid of the parentheses in the calculation of $(B * 10^{k1})$. This will automatically equal 0 every time.

- 4.5 For variables **K1** and **K2**, we will use byte 30 which is c0. According to the IPMI specification, we split the bit in half. **K2** is the first half, and **K1** is the second half. As stated prior, we do not need **K1** due to the multiplication of **B** being zero. All we need for this program for our voltage readings is **K2**. **K2** is C.
According to the specification, we must use the two's complement theory. Let us take the hex(C) and convert it to binary *1100*. The sign of the integer is the first bit, 0 being positive, and 1 being negative. So our number is negative.

Then we take a binary number and invert all the bits; *1100* becomes *0011*.
Following the two's complement theory, if the number is negative, we add 1 to the number. $0011 + 1 = 0100 = 4$. Since our number is negative now flip the sign.

$$K2 = -4$$

- 4.6 Now that we have all our variables, we can apply the math to our updated equation below. For raw data X, I will use 194, this a normal number that the sensor will receive for 5 volts.

$$y = 261(194) * 10^{-4}$$
$$y \approx 5.06$$

- 4.7 In the program's `get_sensor_data.py` backend script, the convert to volts function handles this logic. I am rounding up to decimal points in order for it to fit in the program's GUI.

```
def convert_to_volts( rawData , multiplier , k2 ):  
    ## take the multiplier and the k2 factors and convert them to volts  
    return round( (( multiplier * rawData) * (10 ** k2)) , 2 )
```

- 4.8 In the program's sensors class, this function is called and passes the proper parameter for each voltage.

```
# 5V  
IPMI_result = run_IPMI(0x02)  
if not IPMI_result: self.vol5v = "!ERROR!"  
else:  
    self.vol5v = convert_to_volts(IPMI_result[1] ,  
                                  261 , -4 )
```

5.0 Temperature

- 5.1 For the temperature, the program simply grabs the raw data, as the temperature is presented in decimal form.

```
# Temp  
IPMI_result = run_IPMI(0x20)  
if not IPMI_result: self.temp = "!ERROR!"  
else: self.temp = IPMI_result[1]
```

- 5.2 For the temperature, the program grabs the raw data, as the temperature is presented in decimal form.

6.0 Power Supplies

- 6.1 For the first part of the power supply, we measure to see if the PSU is plugged into the backplane. If the PSU is plugged into the backplane, then the reading will report back 192. If it does not report back 192, we report error.

```
# PSU1          Hayden Fortunata, 5 days ago • Added wmi ...
IPMI_result = run_IPMI(0x80)
if not IPMI_result: self.psu1 = "!ERROR!"
elif IPMI_result[2] != 192: self.psu1 = "!PSU REMOVED!"
else:
```

- 6.2 To verify the AC plug is in the power supply, we verify that the power out is greater than 11.4. To do this we use the same equation and function discussed in the voltage conversion.

```
else:
    ## PSU status == 192
    IPMI_result = run_IPMI(0x7a)
    volts = convert_to_volts(IPMI_result[1] , 118, -3)
    ## Check if it returns false
    if not volts: self.psu1 = "!ERROR!"
    elif volts < 11.4: self.psu1 = "!AC REMOVED!"
    elif volts > 11.4: self.psu1 = "OK"
```